

Code-Layout, Readability and Reusability

|               |                                                                |
|---------------|----------------------------------------------------------------|
| Date          | 15 July 2026                                                   |
| Team ID       |                                                                |
| Project Name  | SkillEcho - Voice-Based Concept Understanding Analyser (VBCUA) |
| Maximum Marks | 5 Marks                                                        |

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

| S.No | Code Quality Parameter    | Description                                   | Followed (Yes/No/Partial) | Remarks                                                                                                    |
|------|---------------------------|-----------------------------------------------|---------------------------|------------------------------------------------------------------------------------------------------------|
| 1    | Consistent Indentation    | Uniform spacing/tabs used throughout the code | Yes                       | PEP8-style formatting across all service modules                                                           |
| 2    | Proper File Structure     | Files and folders are logically organized     | Yes                       | DDD layout: src/frontend, src/backend/api, src/backend/db, src/backend/services                            |
| 3    | Meaningful Variable Names | Variables reflect their purpose clearly       | Yes                       | e.g. product_info, similarity_score, filler_ratio                                                          |
| 4    | Function / Method Names   | Functions are descriptively named             | Yes                       | transcribe_audio(), extract_signal_features(), evaluate_understanding()                                    |
| 5    | Code Comments             | Inline and block comments explain logic       | Yes                       | Module and function-level docstrings with Args/Returns/Raises                                              |
| 6    | Modular Design            | Code is split into reusable functions/modules | Yes                       | 5 single-responsibility service modules: speech, nlp, audio, scoring, reports                              |
| 7    | No Redundant Code         | Duplicate or unused code is removed           | Yes                       | Shared logic centralised in service modules, imported wherever needed                                      |
| 8    | Error Handling            | Exceptions and errors are handled gracefully  | Yes                       | HTTPException(404) for missing concepts, RuntimeError for model failures, _mark_audio_failed() for cleanup |

Reusable Components / Modules:

| S.No | Component / Module Name | Language / Technology   | Where Reused                                                | Reusability Level (High/Medium/Low) |
|------|-------------------------|-------------------------|-------------------------------------------------------------|-------------------------------------|
| 1    | speech.py               | Python / OpenAI Whisper | Called by the /evaluate pipeline for every audio submission | High                                |

| S.No | Component / Module Name | Language / Technology          | Where Reused                                                                             | Reusability Level (High/Medium/Low) |
|------|-------------------------|--------------------------------|------------------------------------------------------------------------------------------|-------------------------------------|
| 2    | nlp.py                  | Python / Sentence-Transformers | Provides Bi-Encoder similarity, Cross-Encoder accuracy, and Topic Guardrail to /evaluate | High                                |
| 3    | audio.py                | Python / Librosa               | Provides DSP feature extraction and filler-word analysis to /evaluate                    | High                                |
| 4    | scoring.py              | Python (pure logic)            | Combines all metrics into the final 0-100 score for every evaluation                     | High                                |
| 5    | reports.py              | Python / ReportLab             | Generates the downloadable PDF report after every completed evaluation                   | Medium                              |

#### Overall Code Quality Assessment:

| Aspect                   | Rating (1-5) | Comments                                                                                   |
|--------------------------|--------------|--------------------------------------------------------------------------------------------|
| Code Layout & Structure  | 5            | DDD directory structure fully and consistently followed                                    |
| Readability              | 5            | Consistent snake_case/PascalCase naming and full docstring coverage                        |
| Reusability              | 5            | Single-responsibility service modules reused across the entire evaluation pipeline         |
| Documentation / Comments | 5            | Module-level and function-level docstrings throughout, plus structured logging             |
| Overall Score            | 5            | All 15 code-quality criteria in the project's Code Quality Checklist were assessed as Pass |